

The metabolite Markov random field: model, estimation, and why the inference was changed

Notes accompanying the `markov-random-field` code base

Last updated: 14 August 2026

Abstract

This document records the probabilistic model implemented in the code base, an error in the way its parameters were being estimated, the corrections that were made, and the reasoning behind each. It is maintained alongside the code; section 9 lists what is implemented and what is not. A companion file `CONVERSATION.md` records the empirical measurements that motivated each change.

Contents

1	The model	2
1.1	State space	2
1.2	The per-tree factor	2
1.3	The joint field: a product of experts	3
1.4	Observation models	3
2	What was wrong with the estimation	4
2.1	The latent-field updates were correct	4
2.2	The parameter update was not	4
3	Phase 1: maximum pseudo-likelihood on internal nodes	5
3.1	Formulation	5
3.2	Implementation	5
3.3	Result	5
4	Phase 2: leaf conditionals (not yet implemented)	6
4.1	Why it matters	6
4.2	Implementation status and an unresolved discrepancy	6
4.3	The only cross-tree quantity needed	7
5	Exact block sampling of Z	7
5.1	Motivation	7
5.2	The algorithm	7
5.3	Measured outcome: the block on Z alone does not help	8
5.4	Verification by exact enumeration	8
5.5	The right block: (Y, Z) jointly	9
5.6	What wiring it in still requires	10
6	Evidence and controls	10
7	Identifiability notes	10

1 The model

1.1 State space

There are two trees: a species tree T_s and a molecule tree T_m . Write S for the number of species *leaves* and M for the number of molecule leaves.

The observable latent object is the binary matrix

$$Y \in \{0, 1\}^{S \times M}, \quad Y_{ij} = 1 \iff \text{species leaf } i \text{ produces molecule leaf } j.$$

Each tree additionally carries latent states at its *internal* nodes. For the species tree these are

$$Z^s \in \{0, 1\}^{I_s \times M},$$

where I_s is the number of internal species nodes (roots included): for every molecule leaf j there is one binary state per internal species node. Symmetrically $Z^m \in \{0, 1\}^{S \times I_m}$.

Cliques. A *clique* of the species tree is the set of species-tree nodes obtained by fixing a molecule leaf j ; there are M of them, indexed by $c = j$. Within a clique the leaves carry $Y_{\cdot j}$ and the internal nodes carry $Z^s_{\cdot j}$. The cliques of the molecule tree are indexed by species leaves. This is the meaning of the parameter names `species_log_nu_molecules_<j>`: a species-tree parameter indexed by a molecule leaf.

1.2 The per-tree factor

Along each branch of a tree the binary state evolves as a two-state continuous-time Markov chain. For clique c with switching rate $\nu_c > 0$ and stationary occupancy $\alpha_c \in (0, 1)$ the generator is

$$\Lambda_c = \nu_c \begin{pmatrix} -\alpha_c & \alpha_c \\ 1 - \alpha_c & -(1 - \alpha_c) \end{pmatrix}, \quad P_c(t) = e^{\Lambda_c t} = \begin{pmatrix} 1 - \alpha_c + \alpha_c e^{-\nu_c t} & \alpha_c (1 - e^{-\nu_c t}) \\ (1 - \alpha_c)(1 - e^{-\nu_c t}) & \alpha_c + (1 - \alpha_c)e^{-\nu_c t} \end{pmatrix}.$$

The stationary distribution is $(1 - \alpha_c, \alpha_c)$, so α_c is the long-run probability of state 1 and ν_c sets how fast a branch forgets its parent.

Why $e^{\Lambda_c t}$ has a closed form

The exponential above is written out rather than computed, and it is worth recording why that is possible, because the code relies on it: `TTransitionGrid` evaluates the expression directly and links no matrix-exponential routine at all.

Two facts pin the eigenvalues of Λ_c . First, every generator has rows summing to zero — that is what conservation of probability means — so $\Lambda_c \mathbf{1} = 0$ and $\lambda_1 = 0$ is always an eigenvalue, with the constant vector as its eigenvector. Second, a 2×2 matrix has exactly two eigenvalues, and they sum to the trace:

$$\text{tr } \Lambda_c = \nu_c (-\alpha_c - (1 - \alpha_c)) = -\nu_c, \quad \text{hence} \quad \lambda_2 = -\nu_c.$$

(Equivalently $\det \Lambda_c = 0$, which forces one eigenvalue to vanish.)

With two distinct eigenvalues the exponential is a two-term expression in t ,

$$e^{\Lambda_c t} = A e^{\lambda_1 t} + B e^{\lambda_2 t} = A + B e^{-\nu_c t},$$

with A and B independent of t . Two limits fix them. At $t = 0$ the exponential is the identity, so $A + B = I$; as $t \rightarrow \infty$ it tends to the stationary matrix Π_c whose rows are both $(1 - \alpha_c, \alpha_c)$, while $e^{-\nu_c t} \rightarrow 0$, so $A = \Pi_c$. Therefore

$$P_c(t) = \Pi_c + (I - \Pi_c)e^{-\nu_c t}, \quad (1)$$

which is the matrix displayed above, entry by entry.

The economy is worth being explicit about: a general $e^{\Lambda t}$ needs an algorithm — scaling-and-squaring with a Padé approximant, say — that sums a series. Here a single *scalar* exponential $e^{-\nu_c t}$ suffices, and Π_c and $I - \Pi_c$ are constants.

This is a property of the *number of states*, not of this model. Every generator contributes the eigenvalue 0; a two-state generator has only one eigenvalue left over, and the trace determines it. So every two-state chain admits (1). From three states upward the remaining eigenvalues are roots of a cubic or higher, may be complex or repeated, and a numerical routine is generally needed again.

Branch lengths are discretised onto a grid of K bins. With $\delta = 2/(K + 1)$, bin k represents length $\delta(k + \frac{1}{2})$, and the code builds the bin matrices by the recursion $P^{(0)} = e^{\Lambda_c \delta/2}$, $P^{(k)} = P^{(k-1)}e^{\Lambda_c \delta}$ (`TTransitionGrid`). The bin indices are constrained so that the mean branch length is 1, which is what makes ν_c identifiable at all: only the product $\nu_c t$ enters P .

Note that (1) is applied only to the two base matrices, at $\delta/2$ and at δ ; the rest of the grid is still reached by the recursion above, and *not* by evaluating (1) at each $\delta(k + \frac{1}{2})$ in turn. That is deliberate. The independent Python reference used for validation does evaluate per bin, so the two implementations reach the same matrices by different routes and a discretisation error in either one is exposed rather than mirrored. See `docs/adr/0003`.

For a single tree, writing x_v for the state at node v , $\text{pa}(v)$ for its parent and t_v for the length of the branch above it, the factor is

$$f_s(Y, Z^s | \theta_s) = \prod_{c=1}^M \left[\pi_c(x_{r_c}) \prod_{v \neq r_c} P_c(t_v)_{x_{\text{pa}(v)}, x_v} \right], \quad (2)$$

where r_c is the root and $\pi_c = (1 - \alpha_c, \alpha_c)$. Taken on its own, (2) is a *proper, normalised* distribution over (Y, Z^s) : $\sum_{Y, Z^s} f_s = 1$.

1.3 The joint field: a product of experts

The two trees are combined multiplicatively, sharing the leaf layer Y :

$$p(Y, Z^s, Z^m | \theta) = \frac{f_s(Y, Z^s | \theta_s) f_m(Y, Z^m | \theta_m)}{C(\theta)}, \quad C(\theta) = \sum_Y p_s(Y) p_m(Y), \quad (3)$$

where $p_s(Y) = \sum_{Z^s} f_s$ is the species tree's marginal over the leaf layer, and likewise for p_m .

This is the crux. Each factor is separately normalised, but their product is not: Y is claimed by both. The constant $C(\theta)$ is a function of every α, ν and branch length in *both* trees, and it is a sum over 2^{SM} configurations — intractable.

Note that $C(\theta)$ becomes independent of θ_s exactly when p_m is uniform over Y ; that special case is used in section 6 as a control.

1.4 Observation models

Two data sources sit above Y (a third, mass spectrometry, is stubbed out).

LOTUS. $L_{ij} \in \{0, 1\}$ records whether an occurrence was reported. With $o_i^s = \log(1 + \text{papers}_i)$ and likewise o_j^m , the detection probability is

$$\rho_{ij} = (1 - e^{-\gamma_s o_i^s})(1 - e^{-\gamma_m o_j^m}), \quad \Pr(L_{ij} = 1 | Y_{ij}) = \begin{cases} \rho_{ij} & Y_{ij} = 1, \\ \varepsilon & Y_{ij} = 0. \end{cases}$$

Simple error model. D_{ij} reports Y_{ij} flipped independently with probability $\varepsilon_{\text{simple}}$.

2 What was wrong with the estimation

2.1 The latent-field updates were correct

$C(\theta)$ cancels in any *conditional* of (3), so the Gibbs updates for the field were already right:

$$\Pr(Y_{ij} = y | \text{rest}) \propto P_j^s(t_i)_{Z_{\text{pa}(i),j}^s, y} \cdot P_i^m(t_j)_{Z_{i,\text{pa}(j)}^m, y} \cdot \Pr(\text{data}_{ij} | y), \quad (4)$$

$$\Pr(z_v = s | \text{rest}) \propto P_c(t_v)_{z_{\text{pa}(v)}, s} \prod_{u \in \text{ch}(v)} P_c(t_u)_{s, x_u}. \quad (5)$$

Two structural consequences of (4) are worth stating explicitly.

- (a) The conditional of Y_{ij} depends on the rest of the field *only* through the two parent states $Z_{\text{pa}(i),j}^s$ and $Z_{i,\text{pa}(j)}^m$. Hence **all Y cells are conditionally independent given (Z^s, Z^m)** , and the existing sweep over Y is already an *exact block draw*.
- (b) Given Y , the two trees decouple: $Z^s \perp Z^m | Y$.

So the sampler is a three-block Gibbs scheme on $\{Y\}, \{Z^s\}, \{Z^m\}$.

2.2 The parameter update was not

For θ the constant does *not* cancel. The correct conditional is

$$p(\theta_s | Y, Z^s, Z^m) \propto f_s(Y, Z^s | \theta_s) C(\theta)^{-1} p(\theta_s),$$

but `TTree::update_nu_or_alpha` scored proposals with $\log f_s$ alone. This is a *doubly intractable* posterior being sampled as if it were an ordinary one. Two distinct failures follow.

Bias. The estimate is displaced by $\partial \log C / \partial \theta$. Measured magnitude: ± 0.25 to 0.30 in $\log \nu$ and in α , with the sign depending on the operating point, vanishing entirely when the second tree is made neutral.

An unbounded direction. As $\nu_c \rightarrow 0$ every $P_c(t) \rightarrow I$, so a frozen field drives $\log f_s$ towards its maximum. In the true posterior this is paid for by $-\log C$ growing equally; with C omitted the gain is free. Empirically the field term gained $\approx 5.6 \times 10^4$ log units between a typical field and a frozen one, and the sampler duly ran $\log \nu \rightarrow -18$, froze Y to all zeros, and charged the entire data set to the error rates.

Why priors cannot patch this. A $\text{Normal}(0, 1)$ penalty at $\log \nu = -7.5$ costs 28 log units; a $\text{Beta}(1, 99)$ penalty at $\varepsilon = 0.14$ costs ≈ 15 . Against 5.6×10^4 these are three orders of magnitude too small. Only a hard constraint (pinning ε) stopped the collapse, which is a diagnostic, not a fix.

3 Phase 1: maximum pseudo-likelihood on internal nodes

3.1 Formulation

Replace the product of unnormalised factors by a product of *normalised full conditionals*. For any subset $\mathcal{V}$ of sites,

$$\log \text{PL}(\theta) = \sum_{v \in \mathcal{V}} \log p(x_v | x_{-v}, \theta). \quad (6)$$

Every term is a probability in $[0, 1]$; $C(\theta)$ appears in both numerator and denominator of each conditional and cancels exactly. Maximum pseudo-likelihood is a composite likelihood and is consistent under the usual regularity conditions, for *any* choice of $\mathcal{V}$ — a fact this implementation exploits.

Phase 1 takes $\mathcal{V} = \{\text{internal nodes of the tree being updated}\}$. By (5) those conditionals involve only that tree’s own transition matrices, so no cross-tree information is required:

$$p(z_v = s | x_{-v}) = \frac{P_c(t_v)_{z_{\text{pa}(v)}, s} \prod_{u \in \text{ch}(v)} P_c(t_u)_{s, x_u}}{\sum_{s' \in \{0,1\}} P_c(t_v)_{z_{\text{pa}(v)}, s'} \prod_{u \in \text{ch}(v)} P_c(t_u)_{s', x_u}}, \quad (7)$$

with $P_c(t_v)_{z_{\text{pa}(v)}, s}$ replaced by $\pi_c(s)$ at a root.

Two things are gained beyond the removal of C :

- (i) The $\nu \rightarrow 0$ direction is no longer unbounded — numerator and denominator move together.
- (ii) The old code scored every Y cell *twice*, once per tree, which over-weighted the field relative to the data by roughly a factor of two. Dropping the leaves removes that double counting outright.

The cost is statistical efficiency: roughly half the nodes, so a standard error worse by about $\sqrt{2}$. Since per-clique ν was already not estimable ($\text{SE} \approx 0.5$ against a true spread ≈ 0.3), little was lost.

3.2 Implementation

The denominator of (7) is exactly the normaliser the Z sampler already computes, so the term is evaluated as a logistic of a log difference,

$$p(z_v = s) = [1 + \exp(\ell_{1-s} - \ell_s)]^{-1}, \quad \ell_s = \log P_c(t_v)_{z_{\text{pa}(v)}, s} + \sum_{u \in \text{ch}(v)} \log P_c(t_u)_{s, x_u},$$

clamping the exponent at 700 so that a vanishing conditional yields $\approx 10^{-305}$ rather than an exact zero (which would send the Hastings ratio to $-\infty$).

Code: `TTree::add_pseudo_likelihood_term`, with `TClique::calculate_log_prob_node_to_children` exposed and given a runtime `UseTryMatrix` flag.

3.3 Result

The field stopped collapsing with the error rates left free. With Y and Z pinned at the truth the estimator became essentially unbiased ($+0.055$ and $+0.024$ in $\log \nu$, against ± 0.25 – 0.30 before).

4 Phase 2: leaf conditionals (not yet implemented)

4.1 Why it matters

After Phase 1, θ learns about ν *only* through the latent Z layer, and Z is itself sampled given θ . This closes a feedback loop: a slightly too small ν produces an over-smoothed Z , which in turn supports a small ν . It is a bias in the target distribution, so no amount of extra sampling removes it.

Measured (true -0.446):

regime	inferred <code>species_mean_log_nu</code>	bias
Y and Z both pinned at truth	-0.402	$+0.055$
Y pinned at truth, Z free	-0.854	-0.400
Y and Z both free	-1.002	-0.546

About three quarters of the residual bias is already present with a *perfect* Y .

Adding the leaf terms anchors θ to Y , which the data constrains directly. The extra terms are, from (4),

$$p(Y_{ij} = y | \text{rest}) = \frac{P_j^s(t_i) Z_{\text{pa}(i),j}^s, y P_i^m(t_j) Z_{i,\text{pa}(j)}^m, y}{\sum_{y'} P_j^s(t_i) Z_{\text{pa}(i),j}^s, y' P_i^m(t_j) Z_{i,\text{pa}(j)}^m, y'}.$$

The other tree’s factor does not depend on θ_s in the numerator but does not cancel from the denominator, which is precisely why it must be carried.

4.2 Implementation status and an unresolved discrepancy

Implemented and **on by default**; `--no_leaf_pseudo_likelihood` restores the internal-nodes-only behaviour of section 3.

Note first that the data likelihood does *not* enter the leaf term. The pseudo-likelihood being maximised is that of the field $p(Y, Z | \theta)$; the factor $\Pr(\text{data} | Y)$ carries no θ and cancels from the Hastings ratio. Only the other tree’s parent term has to be carried, and only because it sits inside the normaliser.

Measured, with Y and Z pinned at the simulated truth:

$\mu_{\log \nu}$ (species)	coupled field (true -0.446)	control: molecules tree neutral (true -0.500)
internal nodes only (Phase 1)	-0.402 (bias $+0.044$)	-0.355 (bias $+0.146$)
with leaf terms (Phase 2)	-0.753 (bias -0.307)	-0.529 (bias -0.029)

On the control the leaf terms behave exactly as the theory says: the estimator improves, on α as well (0.585 versus 0.523 against a true 0.600). On a genuinely coupled field they make it worse.

A diagnostic build that forces the other-tree factor to a constant $\{\frac{1}{2}, \frac{1}{2}\}$ on the coupled scenario gives bias -0.046 , against -0.307 with the real factor. So the factor is live and the leaf-term algebra is right, yet supplying the *correct* factor is worse than supplying a deliberately wrong constant — which cannot be.

The one component the control does not exercise is the cross-tree lookup: when the other tree is neutral its factor is $\{\frac{1}{2}, \frac{1}{2}\}$ regardless of which parent state is read, so a wrong index cannot show up there. That lookup is exactly what matters once the trees couple.

The lookup is correct

`TTree::check_leaf_lookup_consistency()`, run with `--check_leaf_lookup`, recomputes P_{other} for every cell by a route that does *not* assume the clique/leaf role swap: it builds the cell's index in Y space from (clique, leaf), and from that derives the other tree's clique through `get_clique(IndexArray)` — the call the Y sweep itself makes — together with its leaf node and its parent's state.

All 16384 cells agree, for both trees. Mutation-checked, which matters because both test scenarios are 128×128 and a clique/leaf swap would not even go out of range: swapping the roles makes 16256 of 16384 cells disagree (the survivors are exactly the diagonal clique = leaf), and swapping the two dimensions of the Z index makes 5774 disagree.

Then what causes the pinned-field discrepancy?

Not the lookup. The likeliest remaining explanation is that the leaf terms *couple the two trees' parameter estimates*: the species leaf term contains α_m and ν_m , which are themselves weakly identified and wandering. The score for θ_s has zero expectation only when θ_m sits at the truth, so each tree inherits the other's current error — a coupling the internal-node terms do not have, since those involve one tree alone. This is a hypothesis, not a demonstrated fact.

In practice it is dominated by the gain: with an *inferred* field the leaf terms take Y recovery from 0.681 to 0.920 and bring both error rates back to their simulated values.

4.3 The only cross-tree quantity needed

For the species update, cell (i, j) requires the single bit $Z_{i, \text{pa}(j)}^m$. For a fixed species clique (molecule leaf j) that is a fixed molecule-internal index with varying i — a strided walk through Z^m . Transposing those bits once per iteration into a bit-set costs SM bits (≈ 3 MB at $S = M = 5000$) and one $O(SM)$ pass, after which the inner loop is a bit lookup plus one 2×2 matrix.

5 Exact block sampling of Z

5.1 Motivation

Y is already drawn as an exact block. Z is not: the code visits internal nodes one at a time using (5). On a strongly coupled field that produces very long correlation times — measured effective sample size was ≈ 10 over 10^4 post-burn-in iterations, i.e. about one independent draw per 1000 iterations.

5.2 The algorithm

Conditional on Y and θ , each clique's internal states form a two-state tree-structured hidden Markov model, which admits an *exact* joint draw in time linear in the number of nodes.

Upward pass (Felsenstein pruning). For each node v define the partial likelihood of everything observed below v :

$$L_v(s) = \Pr(Y \text{ in the subtree below } v \mid x_v = s) = \prod_{u \in \text{ch}(v)} \left(\sum_{s'} P_c(t_u)_{s, s'} L_u(s') \right),$$

with $L_u(s') = \mathbb{K}[s' = Y_u]$ at a leaf, so that a leaf child contributes simply $P_c(t_u)_{s, Y_u}$. Computed in post-order (children before parents). Working in logs, each L_v is renormalised by its maximum, which is harmless because only ratios are ever used.

Downward pass (backward sampling). At a root,

$$\Pr(x_r = s | Y) \propto \pi_c(s) L_r(s),$$

and thereafter, for an internal node v whose parent has already been sampled,

$$\Pr(x_v = s | x_{\text{pa}(v)}, Y) \propto P_c(t_v)_{x_{\text{pa}(v)}, s} L_v(s).$$

Traversing in reverse post-order (parents before children) yields a draw from the exact joint conditional $p(Z_{\cdot c} | Y, \theta)$.

Cost. $O(\text{number of edges})$ per clique — the same order as a single site-by-site sweep, but returning an independent draw rather than a highly correlated one. The target distribution is unchanged: this is an exact block Gibbs step, not an approximation.

Note the upward pass reads only *leaf* states and transition matrices; it never reads internal states. The downward pass may therefore overwrite internal states in place.

5.3 Measured outcome: the block on Z alone does not help

Implemented and compared against the single-site sweep on identical data and seed. Wall clock was indistinguishable (467 s versus 471 s for 3×10^4 iterations), but:

	block	single-site
ESS of the field density (second half)	9	9
iterations per independent draw	1710	1676
field density, second-half mean	-27489 (sd 7210)	-36660 (sd 4344)
<code>species_mean_log_nu</code> (true -0.446)	-1.569	-1.186

Two things follow. First, **the hypothesis that single-site updating of Z was the mixing bottleneck was wrong**. Second, the two schemes disagree on the posterior although both leave $p(Z | Y, \theta)$ invariant and should therefore share a stationary distribution.

5.4 Verification by exact enumeration

The discrepancy above admits two readings: an implementation error, or a correct sampler reaching regions the single-site sweep could not. These were separated by direct verification.

The algorithm was lifted out of the storage plumbing into a templated free function (`sample_clique_states`) with the tree supplied as a set of callables. The production call site binds them to `TTree/TCurrentState` at no cost, while the tests drive the same code from plain vectors.

For a clique small enough to enumerate, the target is computable in closed form:

$$p(Z | Y) = \frac{\pi_c(x_r) \prod_{v \neq r} P_c(t_v)_{x_{\text{pa}(v)}, x_v}}{\sum_{Z'} \pi_c(x'_r) \prod_{v \neq r} P_c(t_v)_{x'_{\text{pa}(v)}, x'_v}},$$

summing over all $2^{|\text{internal}|}$ configurations. `tests/TUpdate_Z.Tests.cpp` compares this against 2×10^5 draws on a Monte Carlo z -score scale, for a balanced tree, a high switching rate, a low switching rate, a chain, and a root-with-one-leaf case where the root marginal must equal α exactly. All five pass.

The tests were then checked for power by mutation: transposing the parent-to-child lookup in the downward pass fails four of the five, and removing the pruning recursion fails three. (The survivor in the first case is the root-only tree, which has no non-root internal node and therefore cannot exercise that path.)

Verdict. The block sampler is correct. The disagreement in section 5.3 is therefore the *second* reading: better exploration of the field brings the chain closer to a target that is still biased by the Phase-1 pseudo-likelihood (section 4). The apparently worse $\mu_{\log \nu}$ is that bias becoming more visible, not a regression.

5.5 The right block: (Y, Z) jointly

Y was already an exact block and Z now is too, yet nothing changed. This localises the problem to the dependence *between* the blocks: a two-block Gibbs sampler mixes slowly precisely when its blocks are strongly dependent, and here each is close to a deterministic function of the other.

The remedy is to enlarge the block. For a species clique j , condition on the *other* tree only:

$$p(Y_{\cdot j}, Z_{\cdot j}^s | Z^m, \text{data}) \propto \underbrace{\pi_j(x_r) \prod_{v \neq r} P_j^s(t_v)_{x_{\text{pa}(v)}, x_v}}_{\text{species-tree factor}} \cdot \prod_{i=1}^S \underbrace{P_i^m(t_j)_{Z_{i, \text{pa}(j)}^m, Y_{ij}} \Pr(\text{data}_{ij} | Y_{ij})}_{\text{leaf emission}}. \quad (8)$$

Equation (8) is exactly a two-state tree-structured hidden Markov model *with emissions at the leaves*. The recursion of section 5 therefore applies unchanged, except that leaves now carry an emission likelihood instead of a fixed state and are sampled along with the internal nodes.

Two properties make this practical:

- (i) Given Z^m , distinct species cliques share nothing, so all M of them are mutually independent and can be updated in parallel.
- (ii) Both compiled data sources factorise over cells — LOTUS contributes $\Pr(L_{ij} | Y_{ij})$ and the simple error model $\Pr(D_{ij} | Y_{ij})$ — so the emission term is a product of per-cell factors, as (8) requires.

The resulting scheme alternates two exact block draws, $(Y, Z^s) | Z^m$ and $(Y, Z^m) | Z^s$, and attacks the Y – Z dependence directly rather than the within- Z dependence, which has now been measured not to matter.

The generalised recursion

Only one change to section 5 is needed. Where the leaf states were observed, each leaf now carries an *emission likelihood*

$$e_v(s) = \Pr(\text{everything attached to } v | x_v = s),$$

and the leaves are drawn along with the internal nodes. The pruning recursion becomes uniform across node types, with $e_v \equiv 1$ wherever nothing is emitted:

$$L_v(s) = e_v(s) \prod_{u \in \text{ch}(v)} \left(\sum_{s'} P_c(t_u)_{s, s'} L_u(s') \right), \quad (9)$$

so a leaf, having no children, is simply $L_v = e_v$. The downward pass is unchanged except that it now visits every node, drawing a leaf from $P_c(t_v)_{x_{\text{pa}(v)}, s} e_v(s)$.

Comparing (9) with (8), the emission for species leaf i in clique j is

$$e_i(s) = \underbrace{P_i^m(t_j)_{Z_{i, \text{pa}(j)}^m, s}}_{\text{other tree}} \cdot \underbrace{\Pr(L_{ij} | s) \Pr(D_{ij} | s)}_{\text{data}}.$$

That both data sources factorise over cells is precisely what makes this an emission, rather than a term coupling the leaves to one another.

Status. Implemented as `sample_clique_states_with_emissions` and verified the same way as section 5.4, but now enumerating the *joint* $2^{|\text{nodes}|}$ space rather than only the internal nodes: a balanced tree with pseudo-random emissions, a low switching rate, a chain, and a bridge test showing that emissions sharp enough to pin the leaves reproduce the Z -only posterior after marginalisation. All pass; dropping the emission from the pruning pass fails all four. It is *not* yet the production sampler — see section 5.6.

5.6 What wiring it in still requires

- (1) A post-order over *all* nodes. The cached order currently filters the leaves out.
- (2) The other tree’s parent-state bit per cell, $Z_{i,\text{pa}(j)}^m$ — a strided walk through Z^m , cured by the transposed bit-set of section 4.
- (3) Per-cell data likelihoods along a *column* of Y . The two numbers needed are already produced by `TLotus::calculate_LL_update_Y` and `TSimpleErrorModel::probabilities_for_Y_update`, but both are fed from a buffer filled along the last dimension, i.e. across the grain of the species pass.
- (4) Restructuring `TMarkovField`: today it sweeps Y cell by cell and then Z per tree; the joint block replaces both.

Item (3) is shared with Phase 2 (section 4), so the two are best built together.

6 Evidence and controls

The claims above rest on a small number of controlled experiments; they are recorded in full in `CONVERSATION.md`. The most load-bearing:

Neutral-tree control

Setting the second tree to $\alpha = 0.5$ and ν above the stationary threshold makes p_m uniform, hence C independent of θ_s . The bias in $\log \nu$ went from $+0.29$ to -0.012 and in α from $+0.25$ to -0.004 . This is what identifies $C(\theta)$ as the culprit rather than, say, finite-sample bias.

Burn-in control

Re-simulating the field with $15\times$ more iterations changed the bias by 0.001 , excluding simulation non-convergence.

MLE control

A parametric bootstrap on a single-tree CTMC gave a finite-sample bias of -0.044 , an order of magnitude too small to explain the observation.

Pinning ladder

Pinning Y and Z , then only Y , then neither, localises the residual bias to the Z feedback loop (section 4).

7 Identifiability notes

Per-clique ν . From n branches the standard error of $\widehat{\log \nu}$ is ≈ 0.53 at $n = 254$. Simulations with a true spread of $\text{sd} \approx 0.09$ therefore cannot be recovered: the attainable correlation between truth and estimate is ≈ 0.17 . This is a property of the design, not a defect of the code, and it argues for sharing ν across cliques (or making it a function of covariates) rather than estimating M separate values.

γ . Detection saturates: with $\log(1 + \text{papers})$ of order 1.6 , the factor $1 - e^{-\gamma o}$ exceeds 0.997 once $\gamma \gtrsim 5$, so the likelihood is flat above that. Simulating at $\gamma = 5$ places the truth inside the flat region. A $\text{Gamma}(2, 4.6)$ prior has mean ≈ 0.43 , which puts detection at $\frac{1}{2}$ for the median paper count and negligible mass on the saturated corner.

ν versus branch lengths. Only the product $\nu_c t$ enters P . Identifiability comes solely from the constraint that the mean binned branch length is 1.

8 Simulation-side corrections

Variance versus standard deviation. `model_validation/src/params.py` drew $\log \nu_c \sim \mathcal{N}(\mu, \text{var_log_nu})$ by passing the variance to `numpy`’s `scale` argument, which is the *standard deviation*. The C++ prior uses `sd =  $\sqrt{\text{var}}$` . The simulated spread was therefore the square root of the intended one, and the reported “true” `var_log_nu` was actually a standard deviation. Corrected to `np.sqrt(...)`.

Parameters are drawn from their priors when simulating. Any parameter not fixed from a file is drawn from its prior during `simulate`, not set to the CLI default. Changing a prior therefore changes what gets simulated — which is desirable (one then simulates from the prior one infers under) but shifts the recorded truths between runs.

9 Status

Change	Status	Note
<code>sqrt</code> in <code>params.py</code>	done	Simulator only.
Informative priors on $\gamma, \varepsilon, \mu_{\log \nu}$	done	Fixes γ ; the other two a but do not stop the coll
Phase 1 pseudo-likelihood (internal nodes)	done	Stops the collapse; rem
Block sampling of Z	done, verified	pling bias given a know
		Correct (section 5.4), l
		improve mixing. Kept: i
		free. The old sweep rem
		behind <code>--single_site_</code>
Joint (Y, Z) clique block	recursion done & verified, not wired	Section 5.5. The measur
		tion 5.3 says this, not
		block, is where the mixin
		maining work in section
Phase 2 leaf conditionals	done, on by default	Cross-tree lookup verifie
		Takes Y recovery from
		on the full pipeline. One
		oddity in the pinned-fiel
		tion 4.2.
Shared (non per-clique) ν, α	proposed	Modelling decision; sect